# COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE

# LAB ASSIGNMENT NO. 01

NAME : ALEENA AKHTAR

SAP ID : 69273

SUBMITTED TO : MISS NEHAN SEHAR

DATE : 30TH JAN, 2026

**Riphah International University**

**Gulberg Green Campus**

# ‘QUESTION’

Difference between complier, interpreter and assembler.

# ‘ANSWER’

Computers execute programs only in machine language. Therefore, programs written in high-level or assembly languages must be translated into machine code using language processors such as **compiler**, **interpreter**, and **assembler**. These translators differ in their method of translation and execution.

## COMPILER

A **compiler** translates the **entire high-level language program** into machine code **before execution** and generates an executable file. Errors are reported after compilation, and the compiled program runs faster since translation occurs only once. Compilers are used for large and performance-oriented applications (e.g., C, C++).

## INTERPRETER

An **interpreter** translates and executes a **program line by line** at runtime. It does not create a separate executable file. Errors are detected immediately, but program execution is slower due to repeated translation. Interpreters are commonly used for scripting and debugging (e.g., Python, JavaScript).

## ASSEMBLER

An **assembler** converts **assembly language** into machine language, usually using a one-to-one instruction mapping. Assembly language is machine-dependent, and the resulting code is highly efficient. Assemblers are mainly used in system programming and embedded systems.

# COMPARISON TABLE

| Features | Assembler | Interpreter | Compiler |
|---|---|---|---|
| **Source Language** | Assembly language | High-level language | High-level language |
| **Translation Method** | Instruction by instruction | Line by line | Entire program at once |
| **Output** | Machine code | No separate output file | Executable machine code |
| **Execution Speed** | Very fast | Slow | Fast |
| **Error Detection** | After assembly | One error at a time | After compilation |
| **Reusability** | Executable reused | Must re-interpret every run | Executable reused |
| **Dependency** | Machine-dependent | Platform-independent | Mostly platform-independent |
| **Examples** | MASM, NASM | Python, JavaScript | C, C++, Java |

**Thank You!!!!**